
Synthetic Imaging Validation

A practical guide for SYNTHIA partners

Version 0.1.0 | Documentation edition: September 28, 2026

This guide explains what the repository measures, how to prepare an evaluation and how to interpret the results. It is intended for researchers comparing synthetic medical images or masks with real data, including colleagues who do not routinely work with Python.

For now, this package provides a shared evaluation interface, not a single score that decides whether your synthetic data (SD) is good enough. A useful evaluation combines measurements that answer different questions: intensity accuracy, structural similarity, distribution differences, and mask geometry. The choice depends first on whether a synthetic image has a meaningful real reference. However, we are currently developing some meaningful scores and graphical outputs to improve the readability of the results.

The examples use generic filenames and illustrative arrays. No patient data, trained models, credentials or project-specific storage paths are included.

At a glance

- Inputs: 2D images and 3D volumes, as NumPy arrays, optional PyTorch tensors, NIfTI files or NumPy files.
- Interfaces: a command-line tool for prepared image pairs and a Python API, easier to include in your own repositories.
- Evaluation: image similarity, intensity distributions, binary masks, spatial descriptors, distances between precomputed feature vectors.
- Cohorts: explicit file pairing, manifests, optional label-based summaries and parallel evaluation across pairs.
- Outputs: JSON for further processing, CSV for spreadsheets, PDF for reading, LaTeX for reports.

The software is a research tool, which just evaluates the quality of the SD. It does not establish clinical safety, diagnostic performance, privacy protection or fitness for a particular downstream task.

1. Prepare the evaluation before selecting metrics

Paired and unpaired data answer different questions

A paired comparison assumes that each synthetic image has a corresponding real target on the same spatial grid. This is appropriate, for instance, when evaluating reconstruction against a reference. MAE, SSIM, Dice and surface distances then have a clear interpretation.

For unconditional generation, there is usually no uniquely correct real target. Comparing an arbitrary generated subject with an arbitrary real subject using SSIM mostly measures their anatomical differences. In that setting, compare intensity distributions, mask morphology and suitable feature distributions across cohorts instead.

MS-SSIM includes several spatial scales, but it does not remove the need for meaningful pairing when used as a reference-based accuracy metric. It is not a general replacement for cohort-level evaluation.

Input preparation is the user's responsibility

The package loads and checks prepared data. It does not automatically register, resample, reorient, denoise, clip, normalize or clean masks (preprocessing is highly variable depending on your own data). Small explicit preprocessing helpers are available in the API, but are not a full preprocessing pipeline and are not applied by the CLI.

For paired spatial comparisons, check subject correspondence, shape, orientation, spacing, field of view and alignment. For intensity comparisons, use the same units and scaling on both sides. Background handling and clipping should also follow one protocol.

NIfTI pair loading checks shape and, when both files supply them, spacing and affine metadata. These checks detect incompatible grids and they cannot prove anatomical registration. Disabling spatial checks does not align the images.

--data-range is the width of the intended intensity interval for PSNR, SSIM and MS-SSIM. Use 1 for [0, 1], 2 for [-1, 1], or 20 for an agreed [0, 20] interval. It does not normalize, clip or convert units. You should keep this choice fixed when comparing runs.

Formats and dimensionality

Supported files are .nii, .nii.gz, .npy, and .npz. The CLI expects a single array in an NPZ archive; the loading API can select a named array. DICOM, PNG, and JPEG are not direct inputs. Convert them separately if needed, preserving the intended intensities and geometry.

Scalar images have shape [H, W] or [D, H, W]. Channel and batch axes must be supplied explicitly for SSIM and MS-SSIM. Other numerical metrics may aggregate all array elements. Do not assume that every function returns a separate score per channel or sample. Masks are scalar 2D or 3D arrays, not automatic multiclass segmentations.

2. Install and organise the data

From a checkout of the repository, use a dedicated Python environment and install the package:

```
python -m pip install .
```

For PDF output, include the optional report dependency:

```
python -m pip install "[report]"
```

The core dependencies are NumPy, SciPy, scikit-image, nibabel and tqdm. MS-SSIM uses NumPy/SciPy and does not necessarily require PyTorch. The optional extras are report (ReportLab), viz (Matplotlib), test (pytest, coverage tools and a PDF reader) and torch (the optional TorchMetrics reference backend). Exact version constraints are maintained in pyproject.toml.

Python 3.9 through 3.14 are tested and supported. The CI matrix targets GitHub's ubuntu-latest, windows-latest and macos-latest runners, not every historical release of those operating systems. Python 3.9 is retained for existing environments, but prefer a maintained Python release for a new installation. Optional TorchMetrics compatibility is tested separately.

Option A: matching filenames

```
validation_data/  
  real/  
    case_001.nii.gz  
    case_002.nii.gz  
  synthetic/  
    case_001.nii.gz  
    case_002.nii.gz
```

The directory names are chosen by the user. The recommended pairing rule matches filename stems after removing the image extension. A missing or duplicate match raises an error. Alphabetical pairing is available, but should only be used after checking the ordering carefully.

Option B: a manifest

A CSV manifest is clearer when filenames differ or cases carry labels:

```
case_id,real,synthetic,label  
case_001,real/reference_A.nii.gz,synthetic/generated_01.nii.gz,group_a  
case_002,real/reference_B.nii.gz,synthetic/generated_02.nii.gz,group_b
```

Paths are relative to the manifest's directory unless a base directory is specified. The case_id is an identifier, not a metric input. The label is metadata, not a voxel label map. Names such as group_a and numeric categories such as 0 or 1 are both usable. The manifest template in examples/manifest_template.csv can be copied and edited.

3. Image similarity: compare a prediction with its target

These metrics use matching arrays with comparable intensities. Lower error or higher similarity on their own, do not imply better diagnostic value.

Absolute and squared errors

MAE is the mean absolute pixel/voxel difference, expressed in the input intensity units. It is a good first measure of overall intensity error and is less dominated by large errors than MSE.

MSE is the mean squared difference. It penalises large errors more strongly and has squared intensity units. RMSE is its square root and returns to the original intensity units. All three are zero for identical inputs and increase with disagreement.

NRMSE divides RMSE by a reference scale. The API supports range, mean magnitude and Euclidean/RMS normalisation; the default uses the reference range. It is dimensionless, but is only comparable when the same normalisation is used. A zero reference scale can produce infinity.

PSNR

PSNR expresses the squared-error relationship to a specified data range on a logarithmic scale, in decibels. Higher is better; identical arrays give positive infinity. A larger declared range raises PSNR even if the arrays do not change, so always report the range.

SSIM and MS-SSIM

SSIM compares local luminance, contrast and structure. Identical inputs give 1 and values can also be negative. It is useful alongside intensity errors, but depends on the window, dynamic range, registration and anatomy being compared.

MS-SSIM combines structural comparisons over several downsampled scales. It can capture differences that a single-scale score misses, but may be less sensitive to very small structures. Small inputs use fewer scales and an adjusted kernel. You should report these settings when comparing studies. Tiny inputs that cannot support the local window are rejected.

The native MS-SSIM backend follows the earlier TorchMetrics calculation, with a numerical parity target of 1e-5 on tested inputs rather than bit-for-bit equality. The optional backend remains available for comparisons with older results and may be removed in a future release.

```
from synthetic_imaging_validation import load_pair, mae, ms_ssim

real, synth = load_pair("real.nii.gz", "synthetic.nii.gz")
scores = {
    "mae": mae(real.array, synth.array),
    "ms_ssim": ms_ssim(real.array, synth.array, data_range=1.0),
}
```

Use these scores for aligned targets. Avoid presenting them as generation-quality scores for arbitrarily paired, independent subjects.

4. Intensity distributions: compare values without assuming alignment

Distribution metrics flatten their inputs into intensity samples. They can detect changes in intensity statistics, but lose spatial information (rearranging all voxels can leave the histogram unchanged).

Histograms show where intensities concentrate. Use shared bin edges and consistent background handling. Intensity statistics report count, mean, standard deviation, extrema and selected percentiles. Percentiles can help identify clipping and unusual tails without reducing the comparison to a single number.

Wasserstein-1 measures how far intensities must move to match the other distribution. It is zero for identical distributions, increases with disagreement and uses the same units as the intensities. It does not require equal numbers of intensity samples.

KL divergence is a directional histogram comparison: $KL(\text{real} \parallel \text{synthetic})$ differs from $KL(\text{synthetic} \parallel \text{real})$. Zero indicates matching histograms. The implementation smooths probabilities to avoid division by zero. Results depend on binning and smoothing. They are not a universal quality scale.

Jensen-Shannon divergence is a symmetric histogram comparison, bounded by 0 and 1 with the default base-2 logarithm. Lower means more similar. It is often easier to communicate than KL, but still depends on binning and can miss spatial errors.

`compare_distributions` combines intensity summaries and distribution distances. The API is the appropriate interface for independent cohorts; the CLI remains a paired-file workflow even if only distribution metrics are selected.

```
from synthetic_imaging_validation import compare_distributions

report = compare_distributions(real_intensities, synthetic_intensities)
```

Large voxel counts do not mean large numbers of independent subjects. A pooled histogram can be dominated by background or larger volumes. For scientific comparisons, describe whether subjects are equally weighted and consider subject-level uncertainty estimates outside this package.

5. Masks: overlap, boundaries and morphology

Mask functions threshold values with values $\geq$ threshold, using 0.5 by default. They do not infer what different integer labels mean. For a multiclass label map, explicitly extract a binary mask for each class before evaluating it.

Paired masks

Dice and IoU measure foreground overlap on a common grid. Both range from 0 to 1, with 1 indicating full agreement. They are easy to interpret but depend on object size: a small displacement affects a small lesion more strongly than a large organ. Both return 1 when both masks are empty.

Hausdorff distance measures the largest nearest-boundary discrepancy. It detects isolated errors but is sensitive to outliers. HD95 uses the 95th percentile of the pooled directed boundary distances, which is less sensitive to a single extreme point. You should record this convention, since implementations can differ.

Average surface distance uses the mean of the two directed mean distances. The API also provides mean, median, maximum and percentile summaries through `surface_distance_statistics`. In 2D these are contour distances; in 3D they are surface distances. Two empty masks give zero distance, while one empty mask gives infinity.

Distances are in the supplied spacing units, or pixels/voxels with unit spacing. Millimetres are only meaningful if the supplied spacing is in millimetres. The functions account for axis spacing, not a full sheared affine transform.

The foreground measure ratio is synthetic area or volume divided by reference area or volume, on a common grid. A value of 1 means equal size, not equal shape or location. The older name `volume_ratio` remains an alias. If both masks are empty the ratio is 1; a non-empty prediction against an empty reference gives infinity.

Individual-mask descriptors

Foreground fraction is the proportion of active pixels or voxels. Connected-component statistics describe the number of separate objects, their sizes and the size of the largest component. Connectivity changes which touching pixels count as one object and must be kept consistent.

Component measure distributions give area in 2D or volume in 3D, using spacing when supplied. Dimension-specific area and volume helpers are also available. These descriptors can be compared across independent mask cohorts without inventing paired targets.

Border statistics quantify foreground within an image-edge band. The band width is specified in pixels/voxels, not physical units. Distance-to-border statistics describe how far foreground centres lie from the nearest boundary plane through edge voxel centres. These are crop-dependent descriptors, not anatomical validity checks.

Centroid statistics report index coordinates, normalised coordinates and coordinates scaled by spacing. The latter are relative to the array origin, not full NIfTI world coordinates. Empty masks have no centroid and no defined foreground-to-border distance. `mask_spatial_report` combines these single-mask descriptors.

Learned spatial priors and pipeline-specific valid regions are not implemented.

```
from synthetic_imaging_validation import dice, hausdorff_distance
from synthetic_imaging_validation.metrics.spatial import mask_spatial_report

overlap = dice(synthetic_mask, real_mask)
hd95 = hausdorff_distance(
    synthetic_mask, real_mask, spacing=(1.0, 1.0, 2.0), percentile=95
)
morphology = mask_spatial_report(synthetic_mask, spacing=(1.0, 1.0, 2.0))
```

6. Feature-based comparisons between cohorts

Feature metrics accept matrices shaped [number of samples, number of features]. Both cohorts must use the same encoder and preprocessing, with one comparable feature vector per sampling unit. They do not accept image paths directly. No image encoder, canonical Inception FID, or dedicated medical 3D-FID extraction pipeline is bundled for now. We are trying to get an agreement on the FID encoder to be used.

Frechet feature distance compares the means and covariances of two feature distributions using a Gaussian approximation. Lower indicates greater agreement under that approximation. Results depend strongly on the encoder, sample count, feature dimension and regularisation. It requires at least two samples per cohort, but that mathematical minimum is not a recommendation for reliable estimation.

Kernel Inception Distance (KID) uses an unbiased polynomial-kernel MMD estimator on the supplied features. The name does not imply that Inception features were extracted by this package. Finite-sample estimates can be negative. RBF-MMD uses a Gaussian kernel instead. Its bandwidth affects which differences are emphasised and its unbiased estimate can also be negative.

Feature precision and recall compare local neighbourhoods in feature space. They separate how well synthetic samples fit the real manifold from how much of the real manifold is covered. Report both values, the neighbourhood parameter and cohort sizes. Pairwise distance calculations can be memory-intensive.

Sliced Wasserstein distance compares one-dimensional random projections of the feature distributions. It is useful when a lower-cost distribution comparison is needed, but depends on the number of projections and random seed. Keep these fixed across experiments.

```
from synthetic_imaging_validation import frechet_distance

distance = frechet_distance(real_features, synthetic_features)
```

There is no universal minimum sample count that makes feature scores reliable. In particular, when the feature dimension is comparable to or exceeds the sample count, covariance estimation is unstable. Regularisation makes the calculation possible; it does not supply missing evidence. Small subgroups need the same caution, even when the full cohort is large.

7. Run a study and compare labels

The command-line tool offers three input modes: one file pair, two paired directories, or a manifest. They are alternatives, not options to combine in one run. Metric names are selected explicitly and the default set is MAE, MSE, RMSE, PSNR, SSIM and Wasserstein.

For a single aligned pair:

```
synthetic-imaging-validate --real real.nii.gz --synthetic synth.nii.gz --metrics mae ssim --data-range 1 --output results.json
```

For a manifest with cohort and label summaries:

```
synthetic-imaging-validate --manifest pairs.csv --key-column case_id --group-by label --metrics mae rmse --output-pdf results.pdf
```

Without `--group-by`, a manifest's label column is kept as metadata only. With `--group-by label`, each pair is evaluated first, then scalar results are summarised within each label. This is not a voxel-class segmentation analysis and is not a class-conditional FID calculation.

The Python API also offers `paired_metrics_by_class` for aligned sample arrays and `distribution_metrics_by_class` for independent real and synthetic cohorts with separate label arrays. Explicit class lists can keep expected but absent groups visible. Class-wise output reports sample counts and insufficient-sample conditions.

```
from synthetic_imaging_validation import paired_metrics_by_class

report = paired_metrics_by_class(
    real_batch, synthetic_batch,
    labels=["group_a", "group_b"], metrics=["mae", "rmse"]
)
```

Use `--show-progress` for a pair-level progress bar. Use `--num-workers N` to evaluate up to N pairs concurrently; 1 is sequential and 0 uses the available CPU cores, capped by the number of pairs. This uses threads and preserves result ordering. It does not split one volume across workers or guarantee a speedup. It is advisable to start with a small number and watch memory use.

The API equivalent is `evaluate_pairs(pairs, metric_function, num_workers=N)`. A custom metric must be safe to call concurrently and must not mutate shared arrays or use shared mutable state without synchronisation.

8. Save and interpret results

All selected output formats reuse the same computed results. Exporting a PDF or LaTeX file does not run the metric calculations again.

JSON preserves the nested structure and is the best choice for subsequent processing. CSV uses one row per metric value; cohort runs include pair, summary and optional group rows. PDF provides a paginated reading copy. LaTeX produces an editable standalone .tex document with tables for a technical report.

```
synthetic-imaging-validate --manifest pairs.csv --metrics mae rmse --output-json results.json --output-csv results.csv --output-pdf results.pdf --output-latex results.tex
```

Use `--output` for one file, or combine the format-specific options above. Existing destination files are overwritten; use different filenames or run directories when keeping multiple experiments.

PDF export requires the report extra but no TeX installation. LaTeX export requires no extra Python dependency. Compile it separately with XeLaTeX or LuaLaTeX, or upload it to a compatible editor. User-provided text is escaped rather than executed as LaTeX commands.

The Python API can export any supported result mapping without using the CLI:

```
from synthetic_imaging_validation import write_report

write_report(report, "results.pdf", title="Validation study")
write_report(report, "results.tex", title="Validation study")
```

PDF and LaTeX include supplied per-pair results, metadata, global summaries and grouped summaries. Nested fields are expanded into readable rows, including indexed rows for lists. Values are not rounded by the exporter. No confidence intervals, significance tests or clinical interpretation are added automatically.

In CLI cohort summaries, count is the number of finite scalar observations contributing to that metric. The overall pair count can be larger. std is the population standard deviation, not the standard error. Infinity remains visible in per-pair output but is excluded from cohort statistics; an undefined value is represented as null. A summary mean without its count can therefore be misleading.

Reports may include case identifiers, input paths and manifest metadata. Review those fields before sharing any format externally. The package does not anonymise results. Keep a separate record of preprocessing, metric parameters, package versions and dataset selection: the result files are not a complete experiment provenance record.

9. Checks, limitations and extending the package

Inputs containing NaN or infinity, empty arrays, unsupported dimensions or incompatible paired shapes raise errors rather than being silently repaired. Empty binary masks are a different case and have the documented metric-specific behaviour.

The test suite covers numerical examples, invalid inputs, empty masks, 2D/3D behaviour, loaders, grouping, parallel execution and output formats. CI enforces full statement and branch coverage for the base test suite with its optional testing dependencies. Coverage shows which code paths were exercised; it is not proof of scientific validity, exhaustive numerical correctness or compatibility with every machine and dataset.

Run the base checks after installing the test, plotting and reporting extras:

```
python -m pip install -e ".[test,viz,report]"
python -m pytest -m "not torch" --cov --cov-report=term-missing
```

The repository is modular. New metrics belong in the relevant metrics module and should reuse the existing input checks. Document the units, score direction, assumptions, edge cases and computational cost, then add tests. Contributions should use a short-lived branch and a pull request so that review and CI run before merging into main.

Current limitations include the absence of a full preprocessing pipeline, automatic feature extraction, built-in confidence intervals and Docker packaging. Reports describe the measured comparison; they do not replace visual review, task-specific validation, privacy assessment or clinical evaluation.

10. Experimental scores and validation curves

Two optional scores provide a compact description of image agreement on a 0-100 scale. Higher values mean closer agreement under the chosen protocol, not a probability that an image is clinically correct. These are initial, uncalibrated prototypes; always inspect their component metrics and representative images.

The similarity score gives equal weight to MS-SSIM and one minus MAE divided by a fixed intensity range. Each component is limited to [0, 1], then their mean is multiplied by 100. Use it only for aligned real/synthetic pairs.

The intensity-distribution score gives equal weight to one minus Jensen-Shannon divergence and one minus Wasserstein distance divided by the same fixed range width. Components are again limited to [0, 1].

Jensen-Shannon uses base-2 logarithms and fixed histogram bins. This score measures intensity agreement, not the location of anatomy: rearranging voxels does not change it.

The interval is supplied explicitly, for example `--score-range 0 1` for images already normalized to [0, 1]. No normalization is applied automatically. Keep the interval, histogram bins, validation cases and preprocessing unchanged across comparisons. A wide interval or large shared background can make results look better than they are. Out-of-range and non-finite inputs raise errors.

```
synthetic-imaging-validate --manifest pairs.csv --scores similarity intensity_distribution --score-range 0 1 -  
-output-json results.json
```

Outputs retain score values, normalized components, raw component metrics and protocol settings. Cohort and label summaries average per-case scores with equal case weighting. Pooled cohort distributions are a different comparison.

To follow validation during training, add `--history outputs/history.json --epoch 10 --run-name model_a`. Use the same history path and a new epoch number at each checkpoint. The history stores global per-case means, finite observation counts and settings. Duplicate steps and changes to recorded settings within a run are rejected. Only one process should write a given history file.

After installing the viz extra, add `--plot-history outputs/history.png` to draw curves, or `--plot-output results.png` for a per-case plot. Select fields with `--plot-metrics mae similarity_score intensity_distribution_score`. Each metric has its own panel. PNG, SVG and PDF figures are supported without a graphical display. Undefined or infinite observations are gaps, not zeros; means exclude them and counts show how many cases contributed.

The Python API offers `similarity_score`, `intensity_distribution_score`, `append_history`, `plot_results` and `plot_history`. Saved results can be plotted without rerunning metric calculations. The runnable `examples/validation_history.py` demonstrates five steps using a controlled intensity offset, not a clinical benchmark.

Further documentation in the repository

- README.md: installation, package overview, CLI and API entry points.
- docs/metrics.md: function-level metric definitions and minimal examples.
- docs/metric_selection.md: choosing metrics and avoiding misleading comparisons.
- docs/data_loading.md: preprocessing expectations, directory pairing, and manifests.
- docs/dimensionality.md: shape conventions, channels and 2D/3D support.
- docs/grouped_metrics.md: class-wise API inputs, outputs and edge cases.
- docs/reporting.md: PDF/LaTeX options, limitations and rebuilding this guide.
- docs/scores_and_plots.md: experimental score formulas, epoch histories and plotting examples.
- CONTRIBUTING.md: contribution workflow and testing expectations.

This document is maintained in docs/validation_guide.md. Its PDF is generated from that source, so revisions do not require editing a binary document manually.